

Practical 12:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 20

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Producer produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        usleep(100000);
    }

    return NULL;
}
```

```

}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumer consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        usleep(150000);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    printf("Buffer size: %d\n", BUFFER_SIZE);
    printf("Items: %d\n\n", ITEMS);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);
}

```

```
pthread_join(producer_thread, NULL);  
pthread_join(consumer_thread, NULL);  
  
sem_destroy(&empty);  
sem_destroy(&full);  
pthread_mutex_destroy(&mutex);  
  
printf("\nProducer-Consumer execution completed.\n");  
  
return 0;  
}
```

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: Trying to lock Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Locked Resource 1\n");

    sleep(1);

    printf("Thread 1: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Trying to lock Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Locked Resource 2\n");

    sleep(1);

    printf("Thread 2: Trying to lock Resource 1\n");
```

```
printf("Thread 2: Locked Resource 1\n");

pthread_mutex_unlock(&resource1);
pthread_mutex_unlock(&resource2);

return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}
```

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 1: Locked Resource 1\n");

    sleep(1);

    printf("Thread 1: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Locked Resource 2\n");

    printf("Thread 1: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 1: Resources released\n");

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Locked Resource 1\n");
```

```

    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 2: Locked Resource 2\n");

    printf("Thread 2: Using both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    printf("Thread 2: Resources released\n");

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    printf("\nDeadlock prevented using resource ordering.\n");

    return 0;
}

```

```

bhanu@DESKTOP-DE32HQ8:~$ cd ~
mkdir -p experiment12
cd experiment12
bhanu@DESKTOP-DE32HQ8:~/experiment12$ nano producer_consumer.c
bhanu@DESKTOP-DE32HQ8:~/experiment12$ gcc -Wall -Wextra -pthread -o producer_consumer producer_consumer.c
producer_consumer.c: In function 'producer':
producer_consumer.c:18:22: warning: unused parameter 'arg' [-Wunused-parameter]
   18 | void *producer(void *arg)
      |                   ^~~~~
producer_consumer.c: In function 'consumer':
producer_consumer.c:40:22: warning: unused parameter 'arg' [-Wunused-parameter]
   40 | void *consumer(void *arg)
      |                   ^~~~~
bhanu@DESKTOP-DE32HQ8:~/experiment12$ ./producer_consumer
Buffer size: 5
Items: 20

Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
Producer produced: 4
Producer produced: 5
Consumer consumed: 4
Producer produced: 6
Consumer consumed: 5
Producer produced: 7
Producer produced: 8
Consumer consumed: 6
Producer produced: 9
Consumer consumed: 7
Producer produced: 10
Producer produced: 11
Consumer consumed: 8
Producer produced: 12
Consumer consumed: 9
Producer produced: 13
Producer produced: 14
Consumer consumed: 10

```

```

Producer-Consumer execution completed.
bhanu@DESKTOP-DE32HQ8:~/experiment12$ #define BUFFER_SIZE 5
bhanu@DESKTOP-DE32HQ8:~/experiment12$ #define BUFFER_SIZE 5
bhanu@DESKTOP-DE32HQ8:~/experiment12$ gcc -Wall -Wextra -pthread -o producer_consumer producer_consumer.c
time ./producer_consumer
producer_consumer.c: In function 'producer':
producer_consumer.c:18:22: warning: unused parameter 'arg' [-Wunused-parameter]
   18 | void *producer(void *arg)
      |                   ^~~~~
producer_consumer.c: In function 'consumer':
producer_consumer.c:40:22: warning: unused parameter 'arg' [-Wunused-parameter]
   40 | void *consumer(void *arg)
      |                   ^~~~~
Buffer size: 5
Items: 20

Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
Producer produced: 4
Producer produced: 5
Consumer consumed: 4
Producer produced: 6
Consumer consumed: 5
Producer produced: 7
Producer produced: 8
Consumer consumed: 6
Producer produced: 9
Consumer consumed: 7
Producer produced: 10
Producer produced: 11
Consumer consumed: 8
Producer produced: 12
Consumer consumed: 9
Producer produced: 13
Producer produced: 14
Consumer consumed: 10

```

```

Consumer consumed: 12
Producer produced: 17
Consumer consumed: 13
Producer produced: 18
Consumer consumed: 14
Producer produced: 19
Consumer consumed: 15
Producer produced: 20
Consumer consumed: 16
Consumer consumed: 17
Consumer consumed: 18
Consumer consumed: 19
Consumer consumed: 20

Producer-Consumer execution completed.

real    0m3.021s
user    0m0.003s
sys      0m0.003s
bhanu@DESKTOP-DE32HQ8:~/experiment12$ nano deadlock.c
bhanu@DESKTOP-DE32HQ8:~/experiment12$ gcc -Wall -Wextra -pthread -o deadlock deadlock.c
deadlock.c: In function 'thread1':
deadlock.c:8:21: warning: unused parameter 'arg' [-Wunused-parameter]
     8 | void *thread1(void *arg)
       |                  ^~~~~~
deadlock.c: In function 'thread2':
deadlock.c:28:21: warning: unused parameter 'arg' [-Wunused-parameter]
    28 | void *thread2(void *arg)
       |                  ^~~~~~
bhanu@DESKTOP-DE32HQ8:~/experiment12$ ./deadlock
Thread 1: Trying to lock Resource 1
Thread 1: Locked Resource 1
Thread 2: Trying to lock Resource 2
Thread 2: Locked Resource 2
Thread 1: Trying to lock Resource 2
Thread 2: Trying to lock Resource 1
^C
bhanu@DESKTOP-DE32HQ8:~/experiment12$ nano deadlock_prevention.c
bhanu@DESKTOP-DE32HQ8:~/experiment12$ ./deadlock_prevention

```